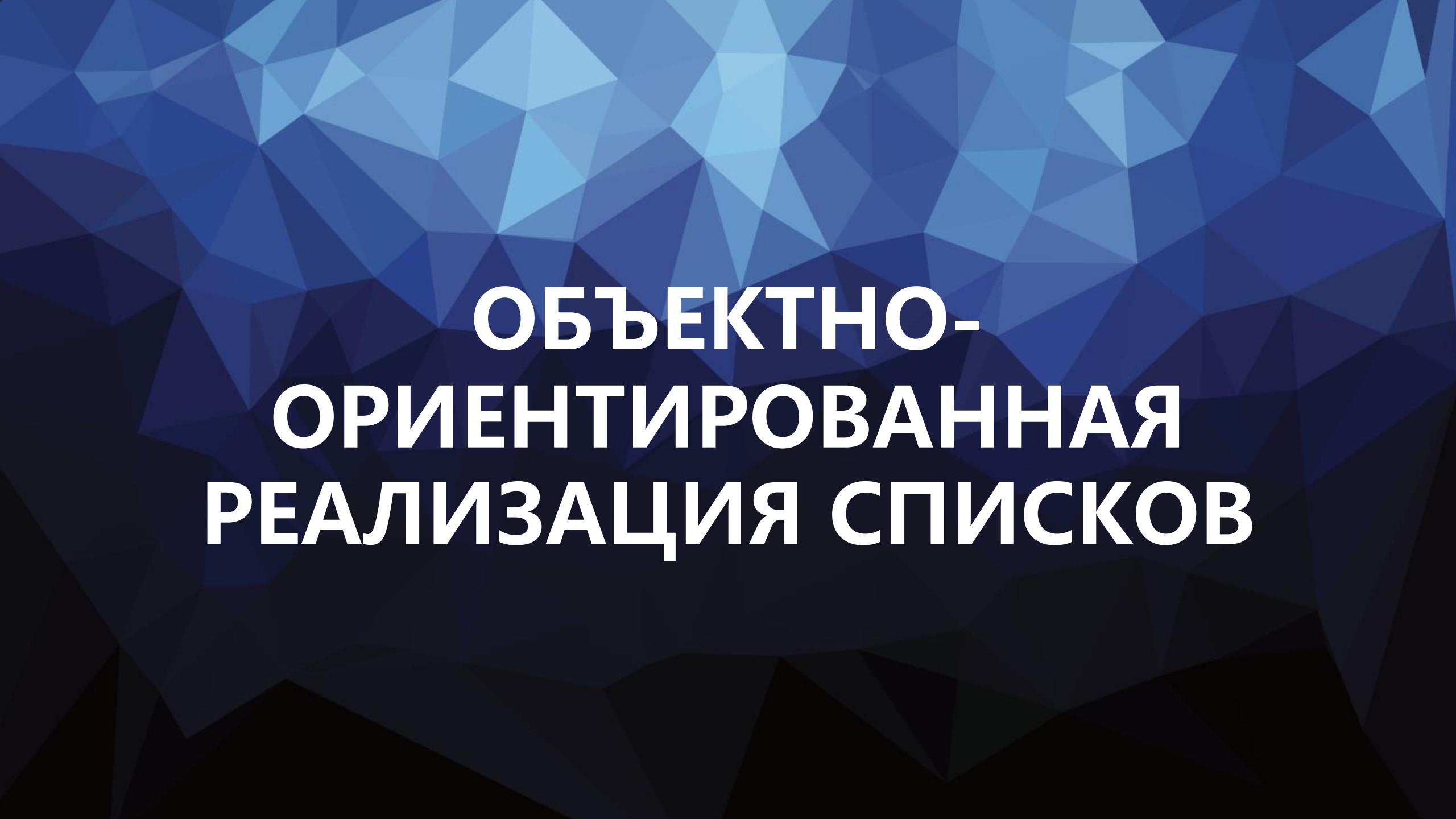




ОБЪЕКТНО- ОРИЕНТИРОВАННАЯ РЕАЛИЗАЦИЯ СПИСКОВ

Основные понятия

Абстрактный тип данных (АТД) — это математическая модель плюс различные операторы, определенные в рамках этой модели. Любой алгоритм может разрабатываться в терминах АТД. Но для реализации алгоритма на конкретном языке программирования необходимо найти способ представления АТД в терминах типов данных и операторов, поддерживаемых данным языком программирования. Для этого используются структуры данных.

В математике список — это последовательность элементов $a_1, a_2, \dots, a_n$ ($n \geq 0$) одного типа, обладающая следующим характеристическим свойством: для любого $i = 1, 2, \dots, n - 1$ элемент a_i предшествует; для любого $i = 2, \dots, n$ a_i следует за a_{i-1} т.е. его элементы линейно упорядочены в соответствии с их позицией в списке.

Основные понятия

Количество элементов n называется длиной списка. Если $n > 0$, то a_1 называется первым элементом списка, а a_n — последним элементом списка. В случае $n = 0$ имеем пустой список, который не содержит элементов.

Однозначно определенного множества операторов для этой модели не существует. В каждом конкретном случае операции со списком будут зависеть от решаемой задачи. Однако можно выделить некоторые базовые операции: добавление элемента в список, удаление элемента из списка, проход по списку (например, для вывода элементов, для подсчета количества элементов с определенным свойством и т.д.).

Список может быть реализован в виде массива или с помощью указателей в виде связного списка

Стек

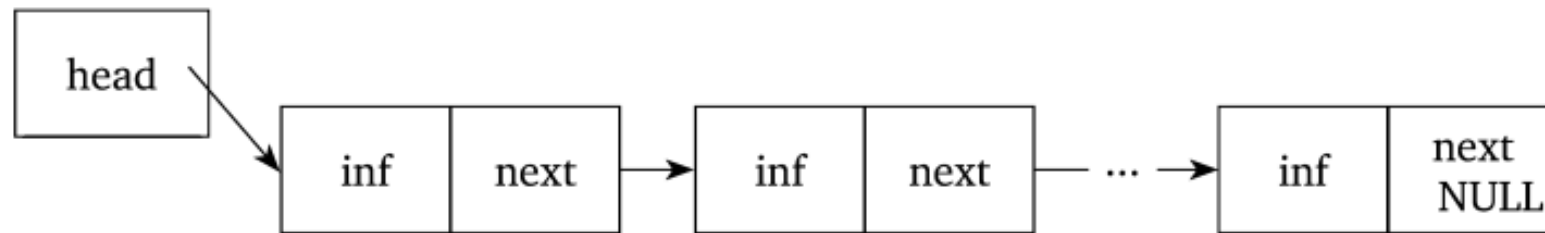
Стек — это частный случай списка, все операции с которым (добавление, просмотр и выборка) выполняются с одного конца, называемого вершиной стека (головой — head). Другие операции со стеком не определены. При выборке элемент исключается из стека. Говорят, что стек реализует принцип обслуживания LIFO (last in — first out, последним пришел — первым вышел). Стек проще всего представить себе в виде пирамиды, на которую надевают кольца.

Достать первое кольцо можно только после того, как будут сняты все верхние кольца.

Стеки широко применяются в системном программном обеспечении, компиляторах, в различных рекурсивных алгоритмах, поэтому очень важно разобраться в принципах работы со стеками.

Стек

Рассмотрим организацию стека с помощью однонаправленного списка:



Базовый элемент стека содержит:

- информационное поле `inf`, которое может быть любого типа, кроме файлового, и будет использоваться для хранения значений, например чисел, символов, строк;
- поле-указатель `next`, в котором хранится адрес следующего элемента в стеке, и которое будет использоваться для организации связи элементов стека.

Стек

Предложенный базовый элемент стека может быть описан следующим образом:

```
struct Element
{
    int inf;           // информационное поле целого типа
    Element *next;    // указатель на следующий элемент стека
    Element(int x, Element *p)
        : inf(x), next(p) // конструктор элемента стека
    {
    }
};
```

Как уже говорилось, доступ к элементам стека возможен только из одной точки — из вершины (головы) стека. Основные операции со стеком — это взять элемент с вершины и положить элемент на вершину. Можно еще добавить операцию просмотра верхнего элемента без его удаления из стека и проверку стека на пустоту.

Стек

С учетом вышеизложенного представим теперь реализацию стека с помощью класса Stack:

```
class Stack
{
    struct Element
    {
        int inf;
        Element *next;
        Element(int x, Element *p) : inf(x), next(p) {}
    };
    Element *head; // указатель на вершину стека
public:
    Stack() : head(0) // конструктор стека
    {}
    bool Empty() // проверка стека на пустоту
    {
        return head == 0;
    }
    int Pop() // взять элемент из стека
    {
        if (Empty()) // если стек пуст, то ничего не делать
        {
            return 0;
        }
    }
```

```
        // иначе запоминаем указатель на вершину стека
        Element *r = head;
        // запоминаем информацию из верхнего элемента
        int i = r->inf;
        // передвигаем указатель стека на следующий от вершины элемент
        head = r->next;
        delete r; // освобождаем память, на которую указывает r
        return i; // возвращаем значение
    }
    void Push(int data) // добавить элемент в стек
    {
        head = new Element(data, head);
    }
    int Top() // посмотреть элемент на вершине стека
    {
        if (Empty()) // если стек пуст, то возвращаем 0
        {
            return 0;
        } else // иначе возвращаем информацию из вершины стека
        {
            return head->inf;
        }
    }
};
```

Стек

Рассмотрим базовые операции со стеком на примерах.

1. Добавление элемента в стек.

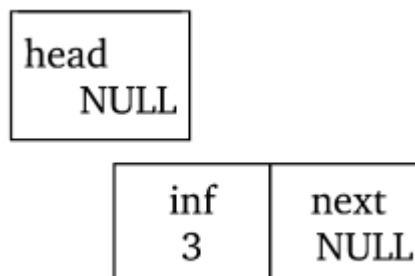
```
void Push(int data)
{
    head = new Element(data, head);
}
```

Теперь последовательно выполним следующие команды:

```
Stack t;    // 1
t.Push(3);  // 2
t.Push(8);  // 3
```

Команда 1 создает экземпляр класса Stack с именем t, при этом вершина стека пуста (поле head равно NULL).

Команда 2 приводит к выполнению оператора head = new Element(3, head). Данный оператор состоит из двух частей:

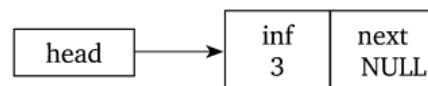


Стек

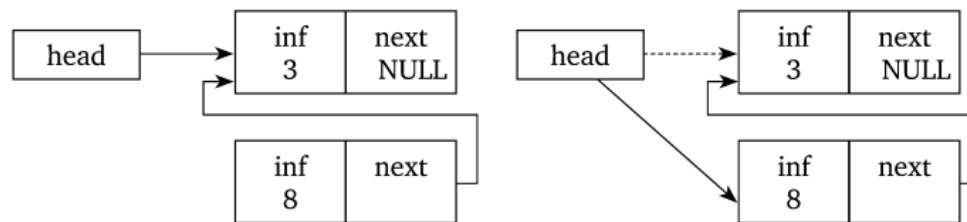
1) выделение памяти под новый элемент списка (new), при этом в поле data нового элемента записывается значение 3, а в поле next записывается адрес области памяти, с которой связан указатель head;

2) переадресация указателя head на новый элемент.

Таким образом, после выполнения команды 2 стек содержит один элемент:



Строка 3 приведет к выполнению оператора `head = new Element(8, head)`, который также последовательно выполнит два действия:



Таким образом, после выполнения команды 2 стек содержит один элемент.



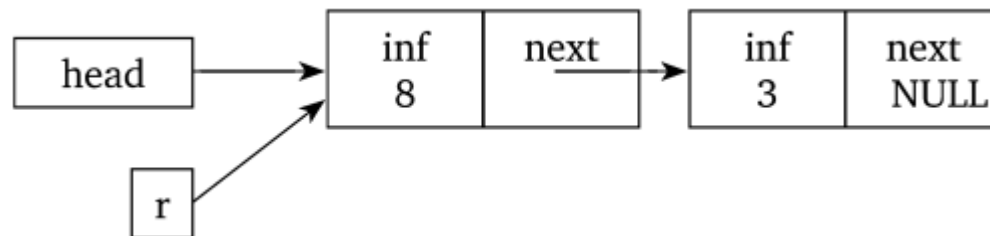
Стек

2. Извлечение элемента из стека.

```
int Pop()
{
    if (Empty()) // если стек пуст, то ничего не делать
    {
        return 0;
    }
    Element *r = head; // 1
    int i = r->inf; // 2
    head = r->next; // 3
    delete r; // 4
    return i; // 5
}
```

Условие пустоты ложное (стек не пуст), поэтому последовательно выполняются следующие операторы.

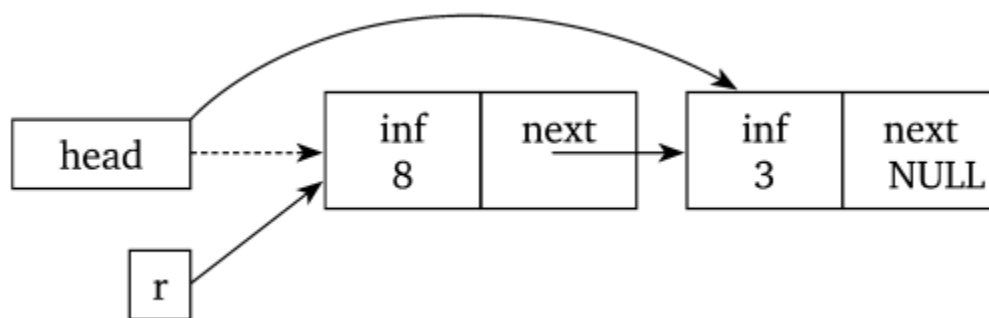
Оператор 1: определяется указатель *r*, который устанавливается на верхний элемент стека.



Стек

Оператор 2: в переменную i записывается 8 — значение информационного поля элемента, на который указывает g .

Оператор 3: указатель $head$ переадресуется на элемент, следующий за g в стеке, таким образом, элемент со значением 8 исключается из стека, а верхним элементом становится элемент со значением 3.



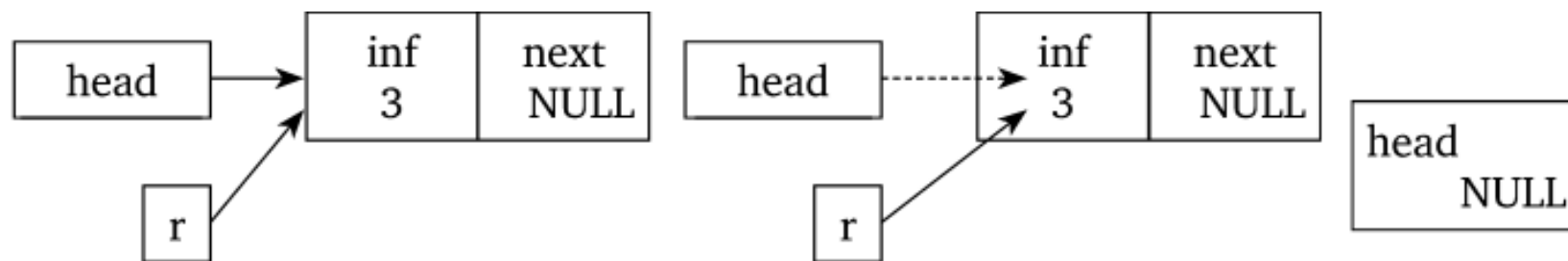
Оператор 4: освобождает область оперативной памяти, связанную с указателем g .



Стек

Оператор 5: функция возвращает значение переменной *i*, т.е. значение 8.

В результате выполнения операции `t.PopO` стек состоит из одного элемента. Повторное выполнение операции `t.PopO` можно проиллюстрировать следующим образом.



Таким образом, стек окажется пустым.

Описание класса `Stack` оформим в виде отдельного файла с именем `stack.cpp`. В дальнейшем при решении практических задач будем подключать данный файл к проекту с помощью директивы `#include "stack.cpp"` после `using namespace std`. Обратите внимание на то, что сам файл `stack.cpp` должен находиться в той же папке, что и файл с кодом программы.

Решение практических задач с использованием стеков

1. Дан текстовый файл `input.txt`, в котором через пробел записаны натуральные числа. Переписать его содержимое в файл `output.txt` в обратном порядке.

```
#include <fstream>
using namespace std;
// подключение файла с реализацией класса Stack
#include "stack.cpp"
int main()
{
    Stack t;
    int i;
    ifstream in("input.txt");
    ofstream out("output.txt");
    while (in >> i) // пока не достигнут конец файла input.txt,
    {               // читаем из него элементы
        t.Push(i); // и помещаем их в стек
    }
    in.close();
    while (!t.Empty()) // пока стек не пуст
```

```
{
    out << t.Pop() << " "; // извлекаем из него элементы
}                          // и выводим в файл output.txt
out.close();
return 0;
}
```

Результат работы программы:

input.txt
1 2 3 4 45 7 4 5 6 7 8 9 10 12

output.txt
12 10 9 8 7 6 5 4 7 45 4 3 2 1

Решение практических задач с использованием стеков

2. Дана последовательность натуральных чисел. Перед каждым элементом, равным x , вставить элемент, равный y . Входная последовательность целых чисел и заданные числа x и y хранятся в файле `input.txt` (в первой строке файла через пробел записаны x и y , во второй — последовательность чисел), выходная последовательность целых чисел записывается в файл `output.txt`.

Идея решения заключается в следующем. Перепишем все числа из файла `input.txt` в стек. По свойству стека все числа там будут находиться в обратном порядке. Нам надо сохранить прямой порядок, а кроме того, вставить новые элементы. Поэтому введем второй стек и будем переписывать числа из первого во второй по следующему алгоритму. Берем число первого стека, переписываем во второй стек. Далее проверяем: если это число x , то во второй стек запишем число y . Таким образом, во втором стеке числа y будут вставлены после чисел x , но при извлечении из стека порядок поменяется на противоположный. Следовательно, остается только извлекать по одному элементу из второго стека и переписывать в выходной файл.

Решение практических задач с использованием стеков

```
#include <fstream>
using namespace std;
// подключение файла с реализацией класса Stack
#include "stack.cpp"
int main()
{
    Stack t, t1; // инициализируем два стека
    int i, x, y;
    ifstream in("input.txt");
    ofstream out("output.txt");
    in >> x;
    in >> y;
    while (in >> i)
    {
        t.Push(i);
    }
    in.close();
    // пока стек t не пуст, переписываем из него числа
    // в стек t1, вставляя после каждого элемента
    // со значением x элемент со значением y
    while (!t.Empty())
    {
        i = t.Pop();
        t1.Push(i);
        if (i == x)
            t1.Push(y);
    }
}
```

```
// переписываем содержимое стека t1 в выходной файл
while (!t1.Empty())
{
    out << t1.Pop() << " ";
}
out.close();
return 0;
}
```

Результат работы программы:

input.txt

4 0
1 4 3 4 45 7 4 6 7 4

output.txt

1 0 4 3 0 4 45 7 0 4 6 7 0 4

Решение практических задач с использованием стеков

3. В файле находится текст, в котором имеются скобки (). Используя стек, проверить, соблюден ли баланс скобок в тексте.

```
#include <fstream>
using namespace std;
#include "stack.cpp"
// Функция, подсчитывающая баланс скобок. Возвращает:
// 0 - если баланс скобок соблюден;
// 1 - если имеется лишняя открывающая скобка;
// -1 - если имеется лишняя закрывающая скобка.
int Balans()
{
    // описываем стек, в который из всего текста будут
    // заноситься только открывающие скобки
    Stack t;
    char i;
    ifstream in("input.txt");
    while (in >> i;)
    {
        switch (i)
        {
            // если считанный символ - (, то записываем его в стек
            case '(': t.Push(i);
                break;
            // если считанный символ ), то в стеке должна находиться
            // соответствующая ему (
            case ')':
                // если стек не пуст, то выбираем из него одну открывающую
                // скобку
                if (!t.Empty())
                {
                    t.Pop();
                }
                // иначе возвращаем -1 - код лишней закрывающей скобки
            else
            {
                return -1;
            }
        }
    }
}
```

```
    }
}
in.close();
/* Если после того, как весь текст обработан, стек пустой, то баланс
скобок в тексте соблюден - возвращаем код 0. В противном случае
в стеке находятся открывающие скобки, которым не соответствуют
закрывающие - возвращаем код 1. */
if (t.Empty())
{
    return 0;
}
else
{
    return 1;
}
}
int main()
{
    ofstream out("output.txt");
    int i = Balans(); // вызываем функцию Balans
    // обрабатываем значение, возвращенное функцией Balans
    if (!i)
    {
        out << "ok";
    }
    else
    {
        if (i == 1)
        {
            out << "error: (";
        }
        else
        {
            out << "error: )";
        }
    }
    out.close();
    return 0;
}
```

Решение практических задач с использованием стеков

Результаты работы программы:

input.txt

```
(5-x*(4/x-(x+3)/(y-2)-y))-9  
(5-x)*(4/x-(x+3)/(y-2)-y)-9  
(5-x)*(4/x-(x+3)/(y-2)-y))-9  
7+9-x/2+4-b*3
```

output.txt

```
error: (  
ok  
error: )  
ok
```

Применение исключений и шаблонов

Извлечь элемент из стека или посмотреть значение верхнего элемента стека можно только тогда, когда стек не пуст. Чтобы обработать указанную ситуацию, мы будем использовать следующий класс:

```
#include <string>
using namespace std;
class StackException
{
private:
    string str;
public:
    StackException(string message) : str(message) {}
    string what()
    {
        return str;
    }
};
```

Данный класс оформим в отдельный файл exception.cpp и подключим его к файлу stack.cpp, в котором предложена реализация класса Stack.

Применение исключений и шаблонов

Аналогичным образом можно создавать классы-шаблоны. Рассмотрим подробно, как будет выглядеть класс Stack после добавления в него механизма обработки исключений и шаблонов:

```
#include "exception.cpp" // подключение класса StackException
// описание класса-шаблона, где Item - тип данных,
// переданный в класс
template <class Item>
class Stack //
{
    struct Element
    {
        Item inf;
        Element *next;
        Element(Item x, Element *p) : inf(x), next(p) {}
    };
    Element *head;
public:
    Stack() : head(0) {}
    bool Empty()
    {
        return head == 0;
    }
    Item Pop()
    {
        if (Empty()) // если стек пуст, то генерируем исключение
        {
            throw StackException("StackException: Pop – стек пуст");
        }
        else // иначе извлекаем элемент из стека
        {
            Element *r = head;
            Item i = r->inf;
```

```
            head = r->next;
            delete r;
            return i;
        }
    }
    void Push(Item data)
    {
        head = new Element(data, head);
    }
    Item Top()
    {
        if (Empty()) // если стек пуст, то генерируем исключение
        {
            throw StackException("StackException: Top – стек пуст");
        }
        else // иначе возвращаем значение верхнего элемента стека
        {
            return head->inf;
        }
    }
};
```

Очередь

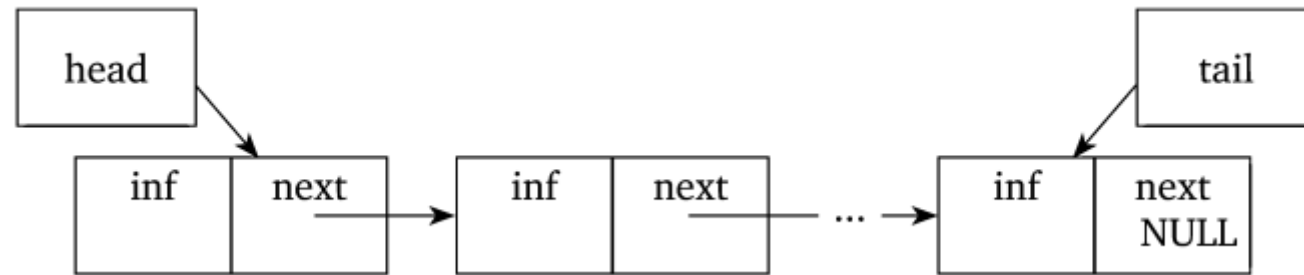
Очередь — это частный случай списка, добавление элементов в который выполняется в один конец (хвост — tail), а выборка и просмотр осуществляются с другого конца (головы — head). Другие операции с очередью не определены. При выборке элемент исключается из очереди. Говорят, что очередь реализует принцип обслуживания FIFO (first in — first out, первым пришел — первым вышел). Очередь проще всего представить в виде узкой трубы, в один конец которой бросают мячи, а из другого конца они вылетают. Понятно, что мяч, который был брошен в трубу первым, первым и вылетит.



В программировании очереди применяются при математическом моделировании, диспетчеризации задач операционной системы, буферном вводе/выводе.

Очередь

Рассмотрим организацию очереди с помощью структуры. Отличие очереди от стека в том, что у очереди два указателя: первый (head) указывает на первый элемент очереди, второй (tail) — на последний элемент.



Базовый элемент очереди содержит:

- информационное поле `inf`, которое может быть любого типа, кроме файлового, и будет использоваться для хранения значений, например чисел, символов, строк;
- поле-указатель `next`, в котором хранится адрес следующего элемента очереди и которое будет использоваться для организации связи элементов.

Очередь

Предложенный базовый элемент очереди может быть описан следующим образом:

```
struct Element
{
    Item inf; // информационное поле типа Item
    Element *next; // указатель на следующий элемент очереди
    Element (Item x) : inf(x), next(0) // конструктор
    {
    }
};
```

Тогда указатели на первый и последний элементы очереди могут быть описаны следующим образом: `Element *head, *tail;`

Как уже говорилось, доступ к элементам очереди возможен в двух точках: из одной точки — (головы) можно брать и просматривать элементы, во второй точке (хвост) можно добавлять элементы.

Очередь

С учетом вышеизложенного представим объектно-ориентированную реализацию очереди с помощью класса Queue:

```
#include "exception.cpp" // подключаем класс QueueException
// рассмотрим его позже
template <class Item> // класс-шаблон очередь
class Queue
{
    struct Element
    {
        Item inf;
        Element *next;
        Element(Item x) : inf(x), next(0) {}
    };
    Element *head, *tail; // указатели на начало и конец очереди
public:
    Queue() : head(0), tail(0) {}
    bool Empty() // возвращает true, если очередь пуста, иначе false
    {
        return head == 0;
    }
    Item Get()
    {
        if (Empty()) // если очередь пуста, то генерируем исключение
        {
            throw QueueException("QueueException: get – queue empty");
        }
        else // иначе извлекаем элемент из головы очереди
        {
```

```
            Element *t = head;
            Item i = t->inf;
            head = t->next;
            if (head == NULL)
            {
                tail = NULL;
            }
            delete t;
            return i;
        }
    }
    void Put(Item data)
    {
        // устанавливаем вспомогательный указатель
        // на последний элемент очереди
        Element *t = tail;
        // формируется новый элемент, на который будет
        // указывать tail
        tail = new Element(data);
        // если до этого очередь была пуста, то новый
        // элемент является и первым, и последним, поэтому
        // указатель head устанавливаем на этот элемент
        if (!head)
        {
            head = tail;
        }
        else
        {
            t->next = tail; // иначе новый узел помещаем в конец очереди
        }
    }
};
```

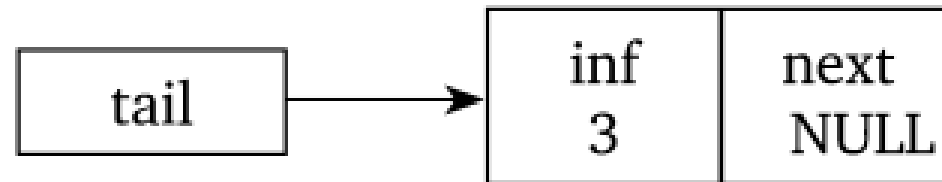
Очередь

Более подробно рассмотрим базовые операции помещения элемента в очередь, предполагая, что у нас создан экземпляр класса `Queue <int> q`. Это значит, что выполнен конструктор, который создал пустую очередь с указателями: `head = NULL`, `tail = NULL`.

Первоначально добавим в очередь элемент 3, для чего выполним команду `q.Put(3)`.

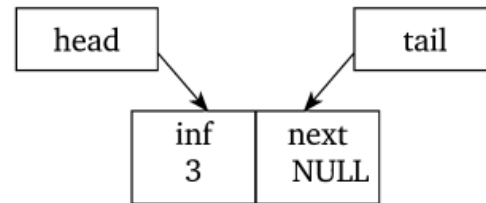
Во вспомогательном указателе `t` сохраняется значение указателя на последний элемент, т.е. `NULL`.

Затем происходит создание элемента, в информационную часть которого заносится значение 3, а в указатель `next` — значение `NULL`. После чего `tail` становится указателем на этот элемент.

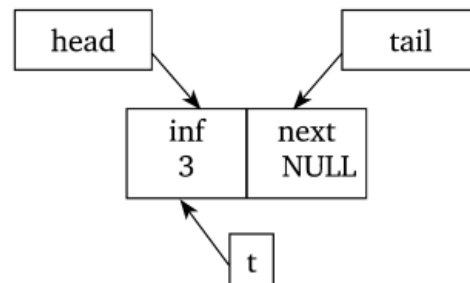


Очередь

Далее производится проверка: пуста ли очередь (указывает ли head на какой-то элемент или его значение NULL)? В данном случае очередь пуста, поэтому head будет указывать туда же, куда и tail, т.е. на новый элемент со значением 3. Таким образом, получили очередь из одного элемента:

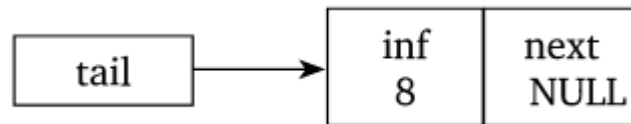


Теперь добавим в очередь элемент со значением 8, выполняя команду `q.Put(8)`. При этом создается вспомогательный указатель, который указывает на последний (в нашем случае единственный) элемент:

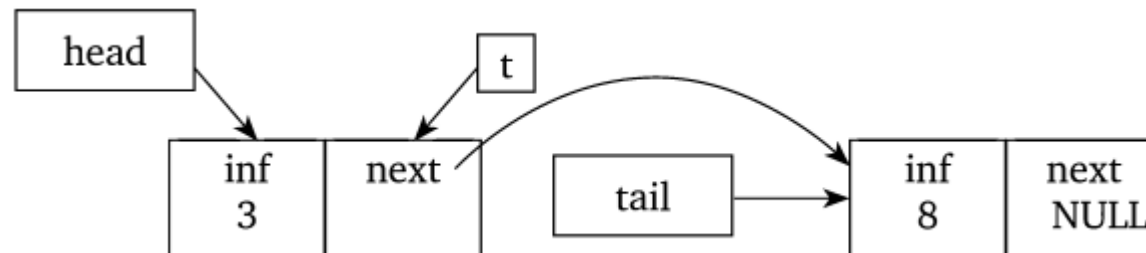


Очередь

Затем происходит создание элемента, в информационную часть которого заносится значение 8, а в указатель next — значение NULL, после чего tail становится указателем на этот элемент:

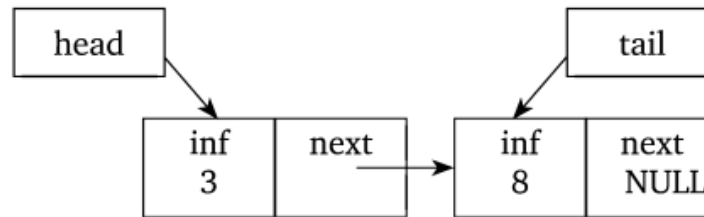


Далее производится проверка: пуста ли очередь (указывает ли head на какой-то элемент или его значение NULL)? В данном случае очередь не пуста, поэтому следующим за элементом, на который указывает t, становится элемент, на который указывает tail.



Очередь

В итоге получаем очередь из двух элементов, которые находятся в очереди в том же порядке, в каком их туда поместили.



Функция Get, которая извлекает верхний элемент из очереди, аналогична функции Put для стека. Отличием является следующий оператор, добавленный в функцию Get:

```
if (head == NULL)
{
    tail = NULL;
}
```

который говорит о том, что если после удаления элемента очередь становится пустой, то и указатель tail нужно «обнулить».

Решение практических задач с использованием очереди

1. Даны файлы data1.txt и data2.txt, компонентами которых являются натуральные числа. Переписать содержимое файла data1.txt в файл data2.txt и наоборот без использования вспомогательного файла.

```
#include <fstream>
#include <string>
using namespace std;
// подключаем файл с реализацией класса-шаблона очередь
#include "queue.cpp"
int main()
{
    Queue<int> t;
    int i;
    // описываем входной поток и связываем его с data1.txt
    ifstream in("data1.txt");
    while (in >> i) // пока не конец файла, считываем из него числ
    {
        t.Put(i); // кладем их в очередь
    }
    in.close(); // закрываем поток, связанный с файлом data1.txt
    // описываем входной поток и связываем его с data2.txt
    ifstream in1("data2.txt");
    // описываем выходной поток и связываем его с data1.txt
    ofstream out("data1.txt");
    // пока не конец файла data2.txt, считываем из него числа
    while (in1 >> i)
```

```
{
    out << i << " "; // записываем в файл data1.txt
}
in1.close(); // закрываем поток, связанный с файлом data2.txt
out.close(); // закрываем поток, связанный с файлом data1.txt
// описываем выходной поток и связываем его с data2.txt
ofstream out1("data2.txt");
// пока очередь не пуста, выбираем из нее элементы
// и записываем в data2.txt
while (!t.Empty())
{
    out1 << t.Get() << " ";
}
out1.close(); // закрываем поток, связанный с файлом data2.txt
return 0;
}
```


Решение практических задач с использованием очереди

Результат работы программы:

— до запуска программы:

data1.txt

7 515 46 46

data2.txt

2 5 8 12 45 6 6

— после запуска программы:

data1.txt

2 5 8 12 45 6 6

data2.txt

7 515 46 46

Решение практических задач с использованием очереди

2. Дана последовательность натуральных чисел. Создать из них очередь и каждый ее элемент, равный x , заменить на элемент, равный y . Входная последовательность целых чисел и заданные числа x и y хранятся в файле `input.txt`, выходная последовательность целых чисел записывается в файл `output.txt`.

```
#include <fstream>
#include <string>
using namespace std;
// подключаем файл с реализацией класса-шаблона очередь
#include "queue.cpp"
int main()
{
    Queue<int> t;
    int i, x, y;
    ifstream in("input.txt");
    ofstream out("output.txt");
    in >> x;
    in >> y;
    // пока файл не пуст, считываем из него очередной элемент
    // если он равен x, то в очередь помещаем y,
    // иначе исходное число
    while (in >> i)
    {
        if (i == x)
        {
            t.Put(y);
        }
        else
        {
            t.Put(i);
        }
    }
}
```

```
in.close();
// пока очередь не пуста,
// извлекаем из нее элементы
// и записываем в выходной файл
while (!t.Empty())
{
    out << t.Get() << " ";
}
out.close();
return 0;
}
```

Результат работы программы:

input.txt

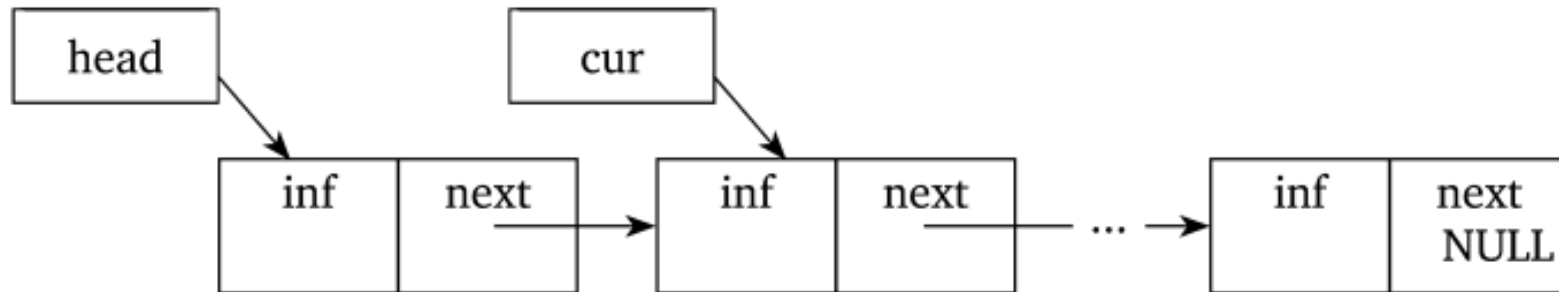
7 0
7 7 1 3 7 5 2 5 7 2 7 9 3 7 7

output.txt

0 0 1 3 0 5 2 5 0 2 0 9 3 0 0

Однонаправленный список общего вида

В общем случае список можно представить в виде линейной связанной структуры с произвольным количеством «точек доступа», что позволяет определить следующие операции над списком: инициализация списка, добавление и удаление элемента из произвольного места списка, поиск элемента в списке по ключу, просмотр всех элементов без их извлечения списка.



Однонаправленный список общего вида

```
#include "exception.cpp"
template <class Item> class List
{
    struct Element
    {
        Item inf;
        Element *next;
        Element(Item x) : inf(x), next(0) {}
    };
    Element *head; // указатель на начало списка
    int size;      // количество элементов в списке
    // возвращает указатель на элемент списка с номером index
    Element *Find(int index) {
        if ((index < 1) || (index > size)) // если индекс элемента списка
        {                                // находится вне диапазона, то
            return NULL;                 // возвращаем NULL
        }
        else                             // иначе
        {
            // устанавливаем указатель на начало списка
            Element *cur = head;
            for (int i = 1; i < index; i++) // и перемещаемся по списку
            { // на элемент с номером index
                cur = cur->next;
            }
            return cur; // возвращаем указатель на требуемый элемент
        }
    }
};
```

```
    }
}
public:
    List()
        : head(0), size(0) // конструктор класса
    {}
    ~List() // деструктор класса
    {
        while (!Empty()) // пока список не пуст
        {
            Remove(1); // удаляем первый элемент списка
        }
    }
    bool Empty() // проверка пустоты списка
    {
        return head == 0;
    }
    int GetLength() // возвращает количество элементов в списке
    {
        return size;
    }
    // возвращает значение элемента списка по его номеру
    Item Get(int index)
    {
        if ((index < 1) || (index > size))
        {
            throw ListException("ListException: get – list error");
        }
        else
        {
            Element *r = Find(index);
            Item i = r->inf;
            return i;
        }
    }
};
```

Однонаправленный список общего вида

```
// осуществляет вставку элемента со значением data в позицию index
void Insert(int index, Item data)
{
    if ((index < 1) || (index > size + 1))
    {
        throw ListException("ListException: insert – list error");
    }
    else
    {
        // создаем новый элемент
        Element *newPtr = new Element(data);
        size = GetLength() + 1; // увеличиваем размерность списка
        if (index == 1) // если вставку производим в позицию 1,
        {
            // то см. рис. 16.6
            newPtr->next = head;
            head = newPtr;
        }
        else // иначе см. рис.16.7
        {
            Element *prev = Find(index - 1);
```

```
            newPtr->next = prev->next;
            prev->next = newPtr;
        }
    }
}
// осуществляет удаление элемента из списка с номером index
void Remove(int index)
{
    if ((index < 1) || (index > size))
    {
        throw ListException("ListException: remove – list error");
    }
    else
    {
        Element *cur; // объявляем вспомогательный указатель
        --size; // уменьшаем размерность списка
        if (index == 1) // если удаляем первый элемент,
        {
            // то см. рис. 16.8
            cur = head;
            head = head->next;
        }
        else // иначе см. рис. 16.9
        {
            Element *prev = Find(index - 1);
            cur = prev->next;
            prev->next = cur->next;
        }
        cur->next = NULL;
        delete cur;
    }
}
// вывод элементов списка в глобальный поток out
void Print()
{
    for (Element *cur = head; cur != NULL; cur = cur->next)
    {
        out << cur->inf << ' ';
    }
    out << endl;
}
};
```

Однонаправленный список общего вида

Более подробно основные операции с однонаправленными списками рассмотрим на примере членов-функций класса List.

1. Проход по списку (например, внутри члена-функции Print). Для реализации этого действия необходим вспомогательный указатель, который вначале устанавливается на первый элемент списка, а потом «пробегаёт» весь список, указывая поочередно на каждый элемент.

Чтобы установить указатель на первый элемент списка, выполняем команду: `Element *cur = head;`

Для вывода содержимого текущего узла в глобальный поток out используется оператор:

```
out << cur->inf;
```

Для перехода на следующий узел используется команда: `cur = cur->next;`

И заканчивается проход по списку тогда, когда указатель сиг примет значение NULL. В результате получаем цикл:

```
for (Element *cur = head; cur != NULL; cur = cur->next)
{
    cout << cur->inf << ' ';
}
```

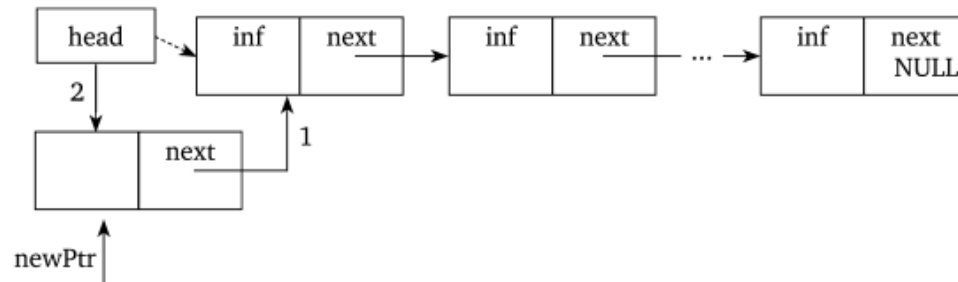

Однонаправленный список общего вида

2. Вставка нового элемента в список в заданную позицию (на примере члена-функции Insert).

Существует два варианта вставки элемента в список.

Вариант первый — это вставка элемента в начало списка. В этом случае необходимо последовательно выполнить две команды:

```
newPtr->next = head; // 1  
head = newPtr;      // 2
```



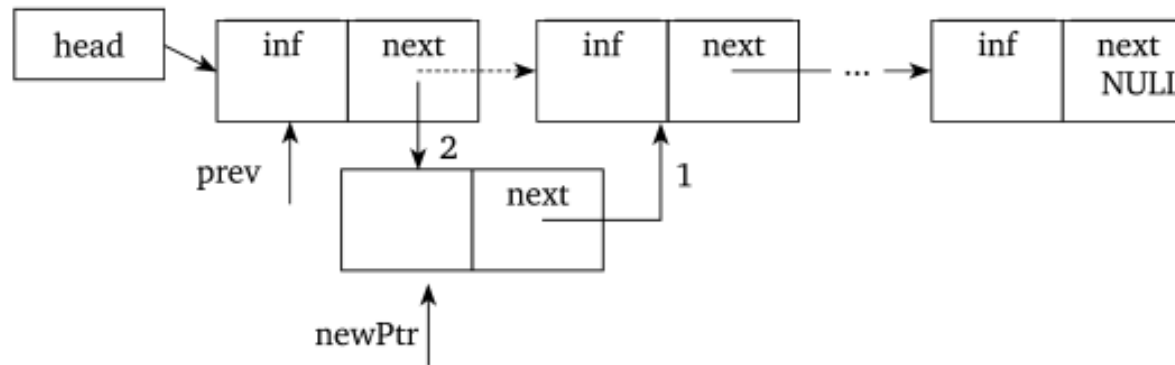
Заметим, что вставка в пустой список — это вставка элемента в начало списка, у которого `head = NULL`. Второй вариант — вставка элемента в середину списка. В этом случае необходимо знать указатель на предшествующий элемент. Узнать его можно с помощью команды:

```
Element *prev = Find(index-1);
```


Однонаправленный список общего вида

Следующие команды позволят нам вставить новый элемент в заданную позицию:

```
newPtr->next = prev->next; // 1  
prev->next = newPtr;      // 2
```



Заметим, что вставка элемента в конец списка пройдет корректно, так как указатель `prev` определен и указатель `prev->next` равен `NULL`.

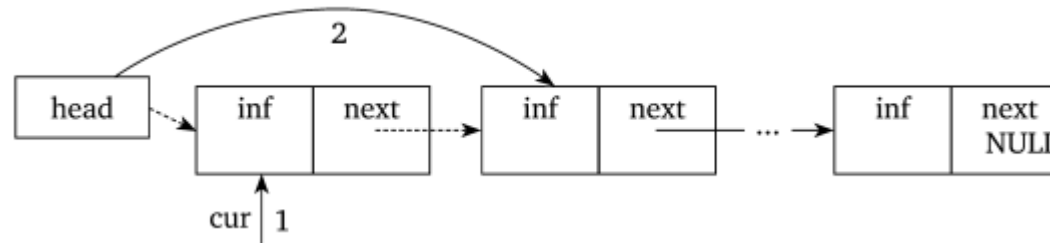
Однонаправленный список общего вида

3. Удаление элемента из списка (на примере члена-функции Remove).

Так же, как и при вставке элемента в список, существует два варианта.

Первый вариант — удаление первого элемента из списка. В этом случае нам необходимо последовательно выполнить две команды:

```
cur = head;           // 1  
head = head->next;    // 2
```



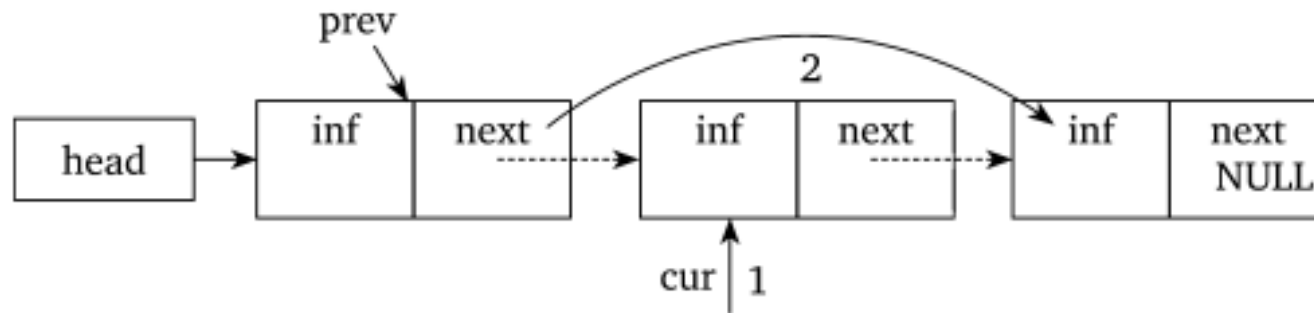
Второй вариант — вставка элемента в середину списка. В этом случае нам необходимо знать указатель на предшествующий элемент. Это можно сделать с помощью команды

```
Element *prev = Find(index-1);
```

Однонаправленный список общего вида

Следующие команды позволят нам удалить требуемый элемент:

```
cur = prev->next;    // 1  
prev->next = cur->next; // 2
```



Заметим, что удаление элемента из конца списка пройдет корректно, так как указатель `prev` определен и указатель `prev->next` равен `NULL`.

При работе со стеком и очередью нам не требовался деструктор, так как обработка стека и очереди подразумевает извлечение из них элементов, в результате чего в конце работы указанные виды списков остаются пустыми. При работе со списками общего вида элементы из них могут и не извлекаться. Поэтому требуется корректно освободить память, выделенную под список, что и делает деструктор.

Решение практических задач с использованием однонаправленных списков

1. Дана последовательность натуральных чисел. Создать из них список и после каждого элемента, равного x , вставить элемент, равный y . Входная последовательность целых чисел и заданные числа x и y хранятся в файле `input.txt`, выходная последовательность целых чисел записывается в файл `output.txt`.

```
#include <fstream>
#include <string>
using namespace std;
// подключаем файл с реализацией класса-шаблона список
#include "list.cpp"
// подключаем глобальные потоки
ifstream in("input.txt");
ofstream out("output.txt");
int main()
{
    List<int> l;
    int value, x, y;
    in >> x;
    in >> y;
    // пока файл не пуст, считываем из него очередной элемент
    // и помещаем в список; позиция, в которую вставляем новый элемент,
    // будет на 1 больше, чем количество элементов в списке
    while (in >> value)
    {
        l.Insert(l.GetLength() + 1, value);
    }
    in.close();
    out << "Исходный список: ";
    l.Print();
    // просматриваем список поэлементно
    for (int i = 1; i <= l.GetLength(); i++) {
        if (l.Get(i) == x) // если значение элемента
            // в i-й позиции равно x
            {
```

```
                {
                    l.Insert(i + 1, y); // вставляем элемент y в эту позицию
                    i++;
                }
            }
    out << "Измененный список: ";
    l.Print();
    l.~List(); // вызываем деструктор
    out.close();
    return 0;
}
```

Результат работы программы:

input.txt

7 0
7 7 1 3 7 5 2 5 7 2 7 9 3 7 7

output.txt

Исходный список: 7 7 1 3 7 5 2 5 7 2 7 9 3 7 7
Измененный список: 7 0 7 0 1 3 7 0 5 2 5 7 0 2 7 0 9 3 7 0 7 0

Решение практических задач с Использованием однонаправленных списков

2. Дана последовательность натуральных чисел. Создать из них очередь и каждый ее элемент, равный x , заменить элементом, равным y . Входная последовательность целых чисел и заданные числа x и y хранятся в файле `input.txt`, выходная последовательность целых чисел записывается в файл `output.txt`. Решение данной задачи можно свести к применению двух членов функций, `Insert` и `Remove`. Так, в функции `main` мы перебираем все элементы списка, если встречаем элемент, равный x , то удаляем его, а на его место вставляем элемент, равный y . Этот алгоритм можно реализовать следующим способом:

```
for (int i = 1; i <= l.GetLength(); i++)  
{  
    if (l.Get(i) == x)  
    {  
        l.Remove(i);  
        l.Insert(i,y);  
    }  
}
```

Решение практических задач с Использованием однонаправленных списков

Однако для решения данной задачи более рационально добавить новую член-функцию Change в класс List. В функции Change мы просматриваем все элементы списка. Если значение очередного элемента совпадает со значением x , то заменяем значение этого элемента на y . Этот алгоритм можно реализовать следующим способом:

```
void Change(Item x, Item y)
{
    // устанавливаем указатель на начало списка
    Element *cur = head;
    // перебираем элементы списка по очереди
    for (Element *cur = head; cur != NULL; cur = cur->next)
    {
        if (cur->inf == x) // если элемент равен x
        {
            cur->inf = y; // заменяем его на y
        }
    }
}
```

Второй способ решения задачи соответствует идеологии ООП, а добавление новой член-функции в класс List существенно расширяет его функциональные возможности.

Решение практических задач с Использованием однонаправленных списков

Теперь полное решение данной задачи выглядит следующим образом:

```
#include <fstream>
#include <string>
using namespace std;
// не забудьте добавить функцию Change в класс List
#include "list.cpp"
ifstream in("input.txt");
ofstream out("output.txt");
int main()
{
    List<int> l;

    int value, x, y;
    in >> x;
    in >> y;
    while (in >> value)
    {
        l.Insert(l.GetLength() + 1, value);
    }
```

```
in.close();
out << "Исходный список: ";
l.Print();
l.Change(x, y); // вызов член-функции Change
out << "Измененный список: ";
l.Print();
l.~List();
out.close();
return 0;
}
```

Результат работы программы:

input.txt

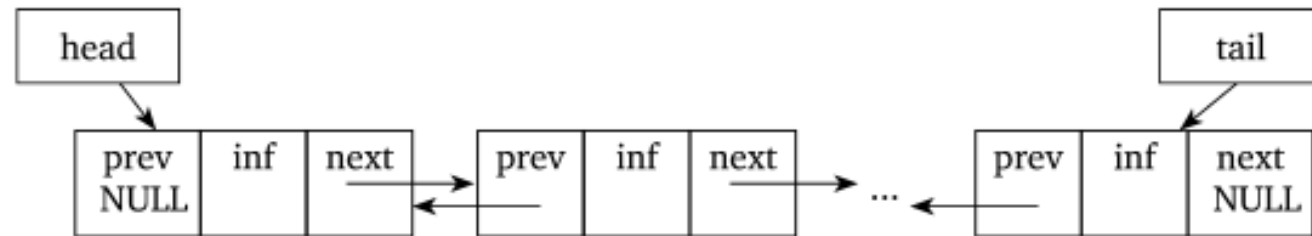
```
7 0
7 7 1 3 7 5 2 5 7 2 7 9 3 7 7
```

output.txt

```
Исходный список: 7 7 1 3 7 5 2 5 7 2 7 9 3 7 7
Измененный список: 0 0 1 3 0 5 2 5 0 2 0 9 3 0 0
```

Двунаправленный список

До сих пор мы рассматривали однонаправленные списки. В таких списках можно передвигаться лишь в одном направлении, а чтобы, например, удалить элемент, необходимо иметь указатель на предыдущий элемент. Однако иногда требуется одинаково эффективно передвигаться по списку в обоих направлениях. В этом случае используют двунаправленные списки, когда в каждом узле имеются два указателя: на следующий и на предыдущий элемент.



Базовый элемент двунаправленного списка содержит:

- информационное поле `inf`, которое может быть любого типа, кроме файлового, и будет использоваться для хранения значений, например чисел, строк, записей;
- поля-указатели `next` и `prev`, в которых хранятся адреса следующего и предыдущего элементов двунаправленного списка соответственно, и которые будут использоваться для организации связи элементов.

Двунаправленный список

Предложенный базовый элемент двунаправленного списка может быть описан следующим образом:

```
struct Element
{
    Item inf;          // информационное поле типа Item
    Element *next;     // указатель на следующий элемент списка
    Element *prev;     // указатель на предыдущий элемент списка
    Element (Item x): inf(x), next(0), prev(0)
    {
    }
}
```

Поскольку в каждом узле такого списка задействовано два указателя вместо одного, механизм операций вставки и удаления будет несколько усложнен по сравнению с односвязными списками. Зато можно удалять элемент, на который есть указатель, и вставлять элементы до и после заданного, не задействуя дополнительные указатели.

Приведем объектно-ориентированную реализацию двунаправленного списка, а затем подробно рассмотрим базовые операции с данным видом списка.

Двунаправленный список

```
#include "exception.cpp"
template <class Item> class DoubleLinkedList
{
    struct Element
    {
        Item inf;
        Element *next;
        Element *prev;
        Element(Item x) : inf(x), next(0), prev(0) {}
    };
    Element *head;
    Element *tail;
    int size;
    // возвращает указатель на элемент списка с номером index
    Element *Find(int index)
    {
        if ((index < 1) || (index > size))
        {
            return NULL;
        }
        else
        {
```

```
            Element *cur = head;
            for (int i = 1; i < index; i++)
            {
                cur = cur->next;
            }
            return cur;
        }
    }
public:
    DoubleLinkedList() : head(0), tail(0), size(0) // конструктор
    {}
    ~DoubleLinkedList() // деструктор
    {
        while (!Empty())
        {
            Remove(1);
        }
    }
    bool Empty() // проверяет список на пустоту
    {
        return head == 0;
    }
    int GetLength() // возвращает количество элементов в списке
    {
        return size;
    }
    // возвращает значение элемента списка по его номеру
    Item Get(int index)
    {
        ... ..
```

Двунаправленный список

```
if ((index < 1) || (index > size))
{
    throw DoubleListException(
        "Exception: get - double-linked list error");
}
else
{
    Element *r = Find(index);
    Item i = r->inf;
    return i;
}
}
// осуществляет вставку элемента со значением data
// слева от элемента с позицией index
void InsertLeft(int index, Item data)
{
    if ((index < 1) || (index > size + 1))
    {
        throw DoubleListException(
            "Exception: insert - double-linked list error");
    }
    else
    {
        Element *newPtr = new Element(data);
        size = GetLength() + 1; // увеличиваем размерность списка
```

```
// устанавливаем указатель на элемент списка
// с заданным номером
Element *cur = Find(index);
// если этот указатель NULL, то список был пуст, поэтому
// новый элемент будет и первым, и последним
if (cur == NULL) // см. рис. 16.11.
{
    head = newPtr;
    tail = newPtr;
} else
// иначе производим вставку в непустой список, при этом
// есть два случая: 1 – частный случай (вставка перед
// первым элементом списка), 2 – общий случай
{
    newPtr->next = cur;
    newPtr->prev = cur->prev;
    cur->prev = newPtr;
    if (cur == head) {
        head = newPtr; // случай 1
    }
    else
    {
        newPtr->prev->next = newPtr; // случай 2
    }
}
}
```


Двунаправленный список

```
// осуществляет вставку элемента со значением data
// слева от элемента с позицией index
void InsertRight(int index, Item data)
{
    if ((index < 1 && head != NULL) || (index > size + 1))
    {
        throw DoubleListException(
            "Exception: insert - double-linked list error");
    }
    else
    {
        Element *newPtr = new Element(data);
        size = GetLength() + 1; // увеличиваем размерность списка
        // устанавливаем указатель на элемент списка
        // с заданным номером
        Element *cur = Find(index);
        // если этот указатель NULL, то список был пуст, поэтому
        // новый элемент будет и первым, и последним
        if (cur == NULL) {
            head = newPtr;
            tail = newPtr;
        }
        // иначе производим вставку в непустой список, при этом
        // есть два случая: 1 - вставка после последнего элемента
        // списка, 2 - вставка в середину списка
        else
        {
            newPtr->next = cur->next;
```

```
            newPtr->prev = cur;
            cur->next = newPtr;
            if (cur == tail)
            {
                tail = newPtr; // случай 1
            }
            else
            {
                newPtr->next->prev = newPtr; // случай 2
            }
        }
    }
}

// осуществляем удаление элемента с номером index из списка
// выделяем четыре случая: 1 - после удаления список становится
// пустым, 2 - удаляем первый элемент, 3 - удаляем последний
// элемент, 4 - общий случай
void Remove(int index)
{
    if ((index < 1) || (index > size))
    {
        throw DoubleListException(
            "Exception: remove - double-linked list error");
    }
    else
    {
        // устанавливаем указатель на заданный элемент
        Element *cur = Find(index);
        --size; // уменьшаем размерность списка
        if (size == 0) // случай 1
        {
            head = NULL;
            tail = NULL;
        }
        else if (cur == head) // случай 2
```


Двунаправленный список

```
{
    head = head->next;
    head->prev = NULL;
}
else if (cur == tail) // случай 3
{
    tail = tail->prev;
    tail->next = NULL;
}
else // общий случай, см.рис. 16.14
{
    cur->prev->next = cur->next;
    cur->next->prev = cur->prev;
}
cur->next = NULL;
cur->prev = NULL;
delete cur;
}
}
```

```
// вывод элементов списка в глобальный поток out в прямом порядке
void PrintLeftToRight()
{
    for (Element *cur = head; cur != NULL; cur = cur->next)
    {
        out << cur->inf << ' ';
    }
    out << endl;
}
// вывод элементов списка в глобальный поток out в обратном порядке
void PrintRightToLeft()
{
    for (Element *cur = tail; cur != NULL; cur = cur->prev)
    {
        out << cur->inf << ' ';
    }
    out << endl;
}
};
```

Двунаправленный список

Более подробно основные операции с двунаправленными списками рассмотрим на примере членов-функций класса `DoubleLinkedList`.

1. Проход по списку в обратном порядке (на примере члена-функции `PrintRightToLeft`).

Для реализации этого действия необходим вспомогательный указатель, который вначале устанавливается на последний элемент списка, а потом «пробегаёт» весь список в обратном порядке по указателям `prev`.

Чтобы установить указатель на последний элемент списка, выполняем команду

```
Element *cur = tail;
```

Для вывода содержимого текущего узла в глобальный поток `out` используется оператор

```
out << cur->inf;
```

Для перехода на следующий узел используется команда: `cur = cur->prev;`

И заканчивается проход по списку тогда, когда указатель `sig` примет значение `NULL`. В результате получаем цикл:

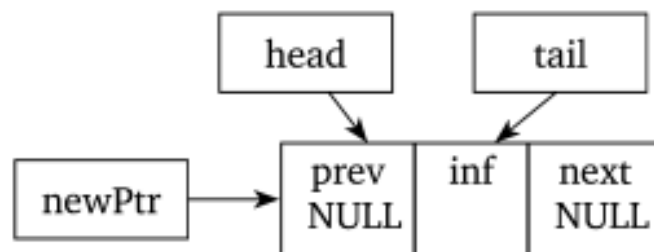
```
for (Element *cur = tail; cur != NULL; cur = cur->prev)
{
    out << cur->inf << ' ';
}
```

Двунаправленный список

2. Вставка нового элемента в список слева от заданной позиции (на примере член-функции InsertLeft).

При вставке нового элемента в двунаправленный список слева от заданной позиции нужно рассмотреть несколько возможных ситуаций.

Если первоначально список пустой, то новый элемент станет одновременно первым и последним. Для этого необходимо установить указатели head и tail на новый элемент:



Двунаправленный список

3. Удаление элемента с заданным номером из двунаправленного списка (на примере функции Remove).

При удалении элемента с заданным номером из двунаправленного списка необходимо рассмотреть четыре случая. Продемонстрируем работу фрагментов функции Remove, отвечающих за обработку каждого из возможных случаев.

Случай 1 (частный) — после удаления список становится пустым:

```
// если после удаления элемента список окажется пустым, то  
// указатели на начало и конец списка «обнулим»  
if (size == 0)  
{  
    head = NULL ;  
    tail = NULL;  
}
```

Случай 2 (частный) — удаляем первый элемент в списке:

```
// если удаляем первый элемент из списка, то указатель  
// head перемещаем на следующий элемент и полагаем,  
// что у этого элемента предыдущего нет  
if (cur == head)  
{  
    head = head->next;  
    head->prev = NULL;  
}
```

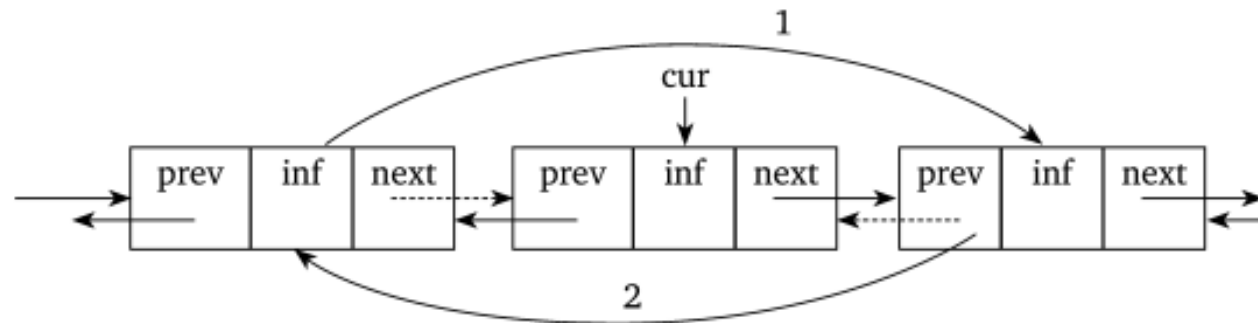
Двунаправленный список

Случай 3 (частный) — удаляем последний элемент в списке:

```
// если удаляемый элемент последний, то указатель tail  
// перемещаем на предыдущий элемент и полагаем,  
// что у этого элемента следующего нет  
if (cur == tail)  
{  
    tail = tail->prev;  
    tail->next = NULL;  
}
```

Случай 4 (общий) — удаляем произвольный элемент в списке:

```
cur->prev->next = cur->next; // 1  
cur->next->prev = cur->prev; // 2
```



Двунаправленный список

Затем, независимо от того, какой именно случай нам встретился, выполняется последовательность команд:

```
// для удаляемого элемента «обнуляем» указатели  
// на следующий и предыдущий элементы, чтобы  
// корректно исключить его из списка  
cur->next = NULL;  
cur->prev = NULL;  
// освобождаем память, связанную с указателем cur  
delete cur;
```


Решение практических задач с Использованием двунаправленных списков

1. Дана последовательность натуральных чисел. Создать из них список, после чего удалить из списка все элементы, равные x . Число x и входная последовательность целых чисел хранятся в файле `input.txt`, выходная последовательность целых чисел записывается в файл `output.txt`.

```
#include <fstream>
#include <string>
using namespace std;
// подключаем файл, содержащий класс-шаблон DoubleLinkedList
#include "double_linked_list.cpp"
// определяем глобальные потоки
ifstream in("input.txt");
ofstream out("output.txt");
int main()
{
    DoubleLinkedList<int> l;
    int value, x;
    in >> x;
    // пока файл не пуст, считываем из него очередной элемент и
    // помещаем в список
    while (in >> value)
    {
        l.InsertRight(l.GetLength(), value);
    }
    in.close();
    out << "Исходный список: ";
    l.PrintLeftToRight();
    // просматриваем список поэлементно
    for (int i = 1; i <= l.GetLength(); i++)
    {
```

```
        if (l.Get(i) == x) // если значение элемента
                           // в i-й позиции равно x
        {
            l.Remove(i); // то удаляем данный элемент из списка
        }
        else
        {
            i++; // иначе переходим к следующему элементу списка
        }
    }
    out << "Измененный список: ";
    l.PrintLeftToRight();
    out.close();
    l.~DoubleLinkedList(); // вызываем деструктор
    return 0;
}
```

Результат работы программы:

input.txt

7
7 7 1 3 7 5 2 5 7 2 7 9 3 7 7

output.txt

Исходный список: 7 7 1 3 7 5 2 5 7 2 7 9 3 7 7
Измененный список: 1 3 5 2 5 2 9 3

Решение практических задач с Использованием двунаправленных списков

2. Дан файл, компонентами которого являются целые числа. Создать на основе данных файла упорядоченную по убыванию структуру данных без использования какого-либо алгоритма сортировки.

```
#include <fstream>
#include <string>
using namespace std;
#include "double_linked_list.cpp"
ifstream in("input.txt");
ofstream out("output.txt");
int main()
{
    DoubleLinkedList<int> l;
    int value, i, k = 1;
    in >> value; // считываем первый элемент из файла
    l.InsertRight(l.GetLength(), value); // помещаем его в список
    // последовательно считываем элементы из файла
    while (in >> value)
    {
        // пропускаем все элементы списка, большие value
        for (i = 1; ((i < k) && (l.Get(i) > value)); i++);
        // если дошли до конца списка и все числа больше value
        if (i == k && l.Get(i) > value)
        {
            // то вставляем value в конец списка
            l.InsertRight(i, value);
        }
    }
```

```
        // иначе нашли элемент в списке, меньший value,
        // и вставляем value слева от этого элемента
        else
        {
            l.InsertLeft(i, value);
        }
        k++; // увеличиваем количество элементов в списке
    }
    in.close();
    l.PrintLeftToRigth();
    out.close();
    l.~DoubleLinkedList();
    return 0;
}
```

Результат работы программы:

input.txt

7 7 1 3 7 5 2 5 7 2 7 9 3 7 7

output.txt

9 7 7 7 7 7 7 5 5 3 3 2 2 1